Received: 15 May 2022 | Revised: 24 September 2022 | Accepted: 27 November 2022

DOI: 10.1002/smr.2532

RESEARCH ARTICLE - EMPIRICAL

Journal of Software: Evolution and Process WILEY

On the diffusion of test smells and their relationship with test code quality of Java projects

Luana Martins [1] | Heitor Costa [2] | Ivan Machado [1]

[1]Institute of Computing, Federal University of Bahia, Salvador, Brazil

[2]Department of Computer Science, Federal University of Lavras, Lavras, Brazil

Correspondence
Luana Martins, Institute of Computing, Federal University of Bahia, Salvador, BA, Brazil.
Email: martins.luana@ufba.br

Funding information
Coordenação de Aperfeiçoamento de Pessoal de Nível Superior, Grant/Award Number: 001; Fundação de Amparo à Pesquisa do Estado da Bahia, Grant/Award Numbers: JCB0060/2016, BOL0188/2020

Abstract

Test smells are considered bad practices that can reduce the test code quality, thus harming software testing goals and maintenance activities. Prior studies have investigated the diffusion of test smells and their impact on test code maintainability. However, we cannot directly compare the outcomes of the studies as most of them use customized datasets. In response, we introduced the `TSSM` (Test Smells and Structural Metrics) dataset, containing test smells detected using the `JNose Test` tool and structural metrics (test code and production code) calculated with the `CK metrics` tool of 13,703 open-source Java systems from GitHub. In addition, we perform an empirical study to investigate the relationship between test smells and structural metrics of test code and the relationship between test smells on a large-scale dataset. We split the projects into three clusters to analyze the distribution of test smells, the co-occurrences among test smells, and the correlation of test smells and structural metrics of test code. The ratio of smelly test classes with a specific test smell is similar among the clusters, but we could observe a significant difference in the number of test smells among them. The test smells *Sleepy Test*, *Mystery Guest*, and *Resource Optimism* rarely occur in the three clusters, and the last two are strongly correlated, indicating that those test smells are more severe than others. Our results point out that most test smells have a moderate correlation with high complexity, large size, and coupling of the test code, indicating that they can also negatively affect its quality. To support further studies, we made our dataset publicly available.

KEYWORDS
Dataset, Replication study, Structural metrics, Test smells

1 | INTRODUCTION

Software testing is a key activity for software quality assurance, which poses challenges for improving the software testing quality.[1] One of those challenges is the presence of test smells in the test code. Test smells have gained importance among the numerous factors that influence the test code quality in recent years.[2–6] Test smells are considered bad practices for developing test code.[7,8] Poorly-written test code can harm the software testing ability to find bugs and, consequently, the software maintenance activities.[9–11]

Researchers have investigated test smells from different perspectives. Some studies have defined test smells, proposed refactoring strategies, and investigated their consequences.[7,8,12–14] Some studies have investigated whether the diffusion of test smells could impact the test code maintainability[11,15] and whether test smells could co-occur together or with code properties (number of classes and lines).[5,16,17] Other studies have investigated the developers' perception of test smells and proposed tools[18–20] and thresholds to analyze them.[10,21,22] Most of those studies use customized datasets, often not shared publicly, making it hard to compare the outcomes as the data come from different projects, versions,

and contexts. In addition, many studies have analyzed test smells at class level, which is done by either representing the existence of a test smell or counting all the occurrences of test smells in the test classes. As the test smells detection occurs in a fine granularity (e.g., method and lines), we argue that the analysis at method level could leverage the relationship between test smells and code properties with more precision.

Listing 1 presents a small excerpt of code of the `LambdaComponentConfigurationTest` test class of the Apache Camel project.* On the one hand, analyzing the test smells at class level results in one entry for the `LambdaComponentConfigurationTest` class with two test smells: *Assertion Roulette* and *Unknown Test*. The first test smell occurs because assertions exist without explanation messages in the `createEndpointWithMinimalConfiguration` method, and the second test smell occurs because the `createEndpointWithoutOperation` method does not contain assertions. On the other hand, analyzing the test smells at method level results in two entries for the `LambdaComponentConfigurationTest` class: i) the `createEndpointWithMinimalConfiguration` method with the *Assertion Roulette* test smell and without the *Unknown Test* test smell, and ii) the `createEndpointWithoutOperation` method with the *Unknown Test* test smell and without the *Assertion Roulette* test smell. As an initial result, the analysis at class level can result in a correlation between the *Assertion Roulette* and *Unknown Test* test smells, but the same does not hold true at method level.

```
public class LambdaComponentConfigurationTest extends CamelTestSupport {

    @Test
    public void createEndpointWithMinimalConfiguration() throws Exception {

        assertEquals("xxx", endpoint.getConfiguration().getAccessKey());
        assertEquals("yyy", endpoint.getConfiguration().getSecretKey());
        assertNotNull(endpoint.getConfiguration().getAwsLambdaClient());
    }

    @Test(expected = IllegalArgumentException.class)
    public void createEndpointWithoutOperation() throws Exception {
        Lambda2Component component = context.getComponent("aws2-lambda", Lambda2Component.class);
        component.createEndpoint("aws2-lambda://myFunction");
    }
```

Listing 1: An example of the detection of the *Assertion Roulette* and *Unknown Test* test smells at class and method levels

Given the difficulty of accessing data from existing datasets and the granularity level of detecting test smells, our goal in this research is twofold: (i) deriving a large-scale dataset to support researchers in studying test smells and comparing their results, and (ii) investigating the relationship between the test smells and the structural metrics of test code at class and method levels. Therefore, we introduce the `TSSM` (Test Smells and Structural Metrics) dataset.† `TSSM` encompasses 19 test smells detected with the `JNose Test`‡,[23] tool and 45 structural metrics of test code and production code calculated with the `CK metrics`§,[24] tool from 13,703 Java open-source projects hosted on GitHub.¶ We used the `TSSM` dataset through a replication study to investigate (i) the diffusion of test smells in JUnit test suites, (ii) the co-occurrences among test smells, and (iii) the association between test smells and structural metrics of test code. Besides, our study analyzed test smells and structural metrics considering different code granularities (at the test class and test method levels). We also analyzed different sizes of test suites, where we grouped the projects into three clusters (projects of 70th, 80th, and 90th percentiles) based on the number of test code lines. Our results can aid developers identify the pairs of test smells that can occur together and the extent of work needed to fix them.

Our findings show that the number of smelly test classes, that is, test classes with at least one test smell, is similar among different sizes of test suites. Still, there is a significant difference in the distribution of test smells. For example, all sizes of test suites have an average of 80% smelly test classes with one *Assertion Roulette* test smell. However, the number of *Assertion Roulette* test smells in the large test suites is two times greater than in the small ones. In addition, the *Sleepy Test*, *Mystery Guest*, and *Resource Optimism* test smells can be more severe than others as they rarely occur in the test suites, and the last two are strongly correlated at class and method levels. Our findings also point out that most test smells have a moderate correlation with high complexity, large size, and high coupling of test classes, indicating that they can negatively affect the code quality.

The remainder of this article is structured as follows. Section 2 presents related work. Section 3 describes the design of our empirical study. Section 4 reports and discusses the results. Section 5 discusses the threats that could affect the validity of our study. Section 6 concludes the paper.

2 | RELATED WORK

The concept of test smells was introduced by van Deursen et al[8] to denote poorly designed test cases encompassing their organization, implementation, and interaction with other test cases. The authors proposed an initial catalog describing test smells and test code refactorings to

handle them. Meszaros et al[7] increased the number of test smells, specifying seven test smells related to the test code level, and five test smells related to the test behavior. Later, several researchers extended the catalog of test smells, investigated the impacts of test smells on the test code quality, and proposed solutions to handle test smells. Next, we present an overview of the effort spent by the research community in recent years to understand test smells and how our proposal differs from the literature.

Several studies have explored different perspectives on the relationship between test smells and software quality. Bavota et al[15] conducted two key empirical studies: the first was an exploratory study on the diffusion of nine test smells in 18 software projects, and the second was a controlled experiment with 20 participants to investigate the effects of test smells on software maintenance. Later, Bavota et al[11] extended both studies by analyzing 27 software projects with an additional 61 participants. Both studies had similar results. The exploratory research presented a diffusion of test smells in 82% and 86% of JUnit tests. The controlled experiment showed that test smells could reduce the comprehension of test code compared to the absence of test smells. Similarly, Peruma et al[12] performed an empirical study on the test smells distribution and survivability on 656 open-source Android apps. The results indicated a widespread occurrence of test smells in apps, which emerge early in their lifetime.

Other studies have investigated the relationship between test smells and structural metrics. Tahir et al[17] conducted an exploratory study with five open-source software projects to investigate the relationship between five structural metrics of production code and nine test smells. The results indicated that the complexity of production code is a "good" indicator of test smells. Virginio et al[25] performed an empirical study with 11 open-source software projects to investigate the relationship between test coverage and 21 test smells. Their results indicated a positive relationship between test smells and test coverage. Pecorelli et al[26] performed an empirical study with 1780 open-source Android apps to assess the tests on those apps and the design of those tests, considering test smells and the structural metrics of test code. Although the authors did not perform a correlation study, the results indicated that the test codes have low quality, considering the test metrics and the test smells.

All the studies above investigated manually written tests, but the production code quality can also influence the test code generation by automated tools. In this direction, Palomba et al[5] analyzed the diffusion of test smells in the test code of 110 open-source software projects generated with the support of the Evosuite tool.[#] The results indicated that 83% of JUnit tests had at least one test smell, similar to the test suites manually written. When comparing different test generation tools, Grano et al[27] studied the influence of production code properties on the generation of smelly test code using automated tools (Randoop,[‖] JTExpert,[**] and Evosuite) in ten open-source software projects. Results showed that the size of the production code and cohesion influence the generation of smelly test code, mainly with the Evosuite tool. Similarly, Virgínio et al[16] investigated the generated test code quality by automated test tools (Randoop and Evosuite) with the existing unit test suite of 21 open-source Java projects regarding the presence of 19 test smells. Results indicated a significant difference in test suite quality, and the existing tests had a smaller distribution of test smells than those generated by tools. Recently, Panichella et al[28] investigated the test smell diffusion in automatically generated test cases and accessed the accuracy of test smell detection tools. The authors built a curated dataset of automatically generated test cases by analyzing 100 Java projects regarding six test smells. They compared it with the output of the `Test Smell Detector`[15] and `tsDetect`[19] tools. The results indicate that the tools are limited and inaccurate in detecting more complex test smells in automatically generated test codes.

Other studies have investigated test smells in different testing frameworks. Baker et al[29] and Zeiss et al[30] identified test smells specific to TTCN-3 test suites and developed the `TRex` tool to make it easier to create and maintain those suites. Later, Counsell et al[31] used the `TRex` tool to explore the trade-offs of TTCN-3 refactorings. Results indicated that considering the dependencies among tests is key for deciding whether refactor a test code. Gatrell et al[32] investigated test refactorings over 270 versions of commercial C# software, and results indicated that base refactorings are common and complex structural refactorings are relatively rare. Bleser et al[33] performed two empirical studies to analyze the diffusion of test smells at class level of 164 open-source SCALA projects and assess the developers' perception of test smells. Results showed that test smells have a low diffusion among test codes, and many developers perceived test smells but did not identify them. Fernandes et al[34] analyzed the strategies for handling test smells in 90 open-source Python projects. As a result, the authors proposed and validated four test smells through a survey with 40 Python developers. Jorge et al[35] performed an empirical study with 11 open-source JavaScript projects to investigate which test smells occur more frequently, whether test smells are likely to occur together, and whether the presence of test smells is related to classical bad design indicators on the test code.

Although those studies explain the relationship between test smells and software quality, other studies pointed out that developers do not always perceive test smells as problematic. Tufano et al[6] surveyed 19 developers from open-source software projects to investigate whether they could recognize test smells in software projects. Besides, they performed an empirical study to investigate the survivability of test smells in 152 open-source software projects belonging to two ecosystems (Apache and Eclipse). The results indicated the developers are unaware of test smells and hardly remove them from the test code. Spadini et al[10] argued that developers could not recognize test smells as problematic due to the lack of thresholds. The authors analyzed 1500 open-source software projects to identify thresholds for nine test smells and evaluated the thresholds with 31 developers from 47 open-source software projects. Results included the definition of non-binary thresholds for four test smells, supporting the user-perceived maintainability impact. Junior et al[9] surveyed 60 practitioners to investigate their awareness of test smells. Results indicated that practitioners introduce test smells during their daily programming tasks. However, the practitioners' experience cannot be

considered a root cause for the insertion of test smells in the test code. Santana et al[36] surveyed 87 practitioners and interviewed eight other practitioners to investigate their perception of test smells and strategies to handle them. Results indicated that most participants consider that test smells should be refactored but do not always do it.

As developers do not prioritize fixing test smells, they have a long lifetime in the projects, which can harm fault detection and maintenance activities. Qusef et al[37] investigated the relationship between the evolution of test smells and faults in the production code. The authors conducted a case study of data collected from 28 versions of Apache Ant. They identified that the absolute number of test smells increases as the project evolves, and some test smells positively correlate with the faults in the production code. Kim et al[38] conducted an empirical study in 12 open-source software projects on the test smell evolution and maintenance. The authors analyzed the commits that removed test smells and concluded the test smells removal was due to maintenance activities. Soares et al[39] investigated how developers refactor existing code to eliminate test smells. The authors surveyed 73 open-source developers to assess their preference and motivation to choose between 10 smelly and refactored test code samples. Next, they submitted 50 pull requests to assess developers' acceptance of the proposed refactorings. The results showed developers preferred the refactored test code for most test smells. In another work, Soares et al[40] investigated whether the testing framework features help refactor existing test code to remove test smells. They conducted a mixed-method study to analyze the usage of the testing framework features in 485 popular Java open-source projects, identifying the features helpful for test smell removal and proposing novel refactorings to fix test smells. The results indicated that 17.6% of the testing framework features are frequently used, limiting optimized propositions to test code creation and maintenance.

Systematic literature mappings and reviews summarize most findings of the related works. Garousi et al[41] performed a multivocal literature mapping on scientific and grey literature to analyze and classify the body of knowledge on test smells. The authors collected data from 120 sources from the industry (e.g., posts in blogs and videos) and 46 sources from academia published until April 2016. The authors presented the largest set of test smells in the literature, including 196 test smells. Additionally, they provided a list of 12 tools and a summary of guidelines, techniques, and approaches to deal with those test smells. Aljedaani et al[42] conducted a systematic mapping to complement the previously mentioned reviews regarding the available tools. The authors selected 47 peer-reviewed papers published until December 2020. Their contributions include a list of 22 tools and comparing them regarding the supported test smells, testing frameworks, and detection strategies (e.g., metrics,[21,43] rules,[18,19,44,45] dynamic tainting,[46,47] and information retrieval[3,44]). In summary, the tools support the detection of 66 test smells, and four tools support the refactoring of 10 test smells among seven different types of testing frameworks.

Given the gap between the developers' perception and the approaches implemented by tools for detecting test smells and refactoring test code, other studies have investigated the feasibility of using machine learning techniques to handle test smells. Martins et al[48] used structural metrics of test code to train machine learning algorithms to classify four test smells. Results indicated that the algorithms performed well in detecting test smells, especially the Random Forest algorithm. Hadj-Kacem et al[49] analyzed the agreement level among the detection tools and suggested a multi-label classification approach to detect test smells based on a deep representation of the test code. Similarly, they found the Random Forest algorithm presented the best results.

Besides investigating the correlation between test smells and structural metrics, most studies aimed to highlight a potential relationship between production code and test code. Differently, we aimed to expand the state-of-the-art practices by providing a deeper understanding of test smells and their relationship with the test code quality. Although previous studies calculated structural metrics to indicate the test code quality, we did not find studies investigating the relationship between test smells and structural metrics of test code. Next, we outlined other related work limitations and how we addressed them.

- **Subject systems.** Most studies analyzed up to 100 Java projects and up to 1780 Android apps. In this study, we analyzed 13,718 open-source Java projects with more than 6 million classes from different domains, from big data processing and warehousing solutions to distributed databases and programming languages.
- **Test smells.** Besides correlating test smells and structural metrics, most studies analyzed up to 11 test smells. The test smells investigated were: *Assertion Roulette*, *Eager Test*, *Mystery Guest*, *Indirect Testing*, *Resource Optimism*, *Sensitive Equality*, *For Testers Only*, *General Fixture*, *Lazy Test*, *Test Code Duplication*, and *Test Run War*.[5,11,15,17,26,27] The test smells detection occurs mainly through the `Test Smell Detector`[15] or `tsDetect`[19] tools by either representing the existence of test smells in the test class or counting all the test smells in the test class. We used the `JNose Test` tool to detect 19 test smells at test classes and test methods. In addition, we presented a categorization of test smells by design problems they can represent in the test code.
- **Structural metrics of test code.** The literature reports the usage of the structural metrics of test code, such as *Weighted Methods per Class* (WMC), *Response for a Class* (RFC), *Lack of Cohesion*, *Tight Class Cohesion* (TCC), and *Loose Class Cohesion* (LCC). However, we did not find studies investigating their relationship with test smells. Therefore, our study brought new insights into the relationship between test smells and test code by analyzing 18 structural metrics o test code to describe its complexity, size, coupling, and cohesion. In addition, we contributed information on the other 31 structural metrics from the test code for further analysis.
- **Structural metrics of production code.** Many studies correlate test smells and systems characteristics such as *Lines of Code* (LOC) and *Number of Classes*.[5,11,15] However, the majority of studies calculated up to seven structural metrics of production code[17,27]: *Coupling Between Objects*

(CBO), *Cyclomatic Complexity*, *Depth Inheritance Tree* (DIT), *Lack of Cohesion of Methods* (LCOM), *Response for a Class* (RFC), *Number of Children* (NOC), and *Weighted Methods per Class* (WMC). Although we did not explore the relationship between test smells and production code, we contributed information on 49 structural metrics of production code for further analysis.

Finally, we contributed to the scientific community by providing the `TSSM` dataset containing 13,718 open-source Java projects with more than 6 million classes, detecting 19 test smells, and collecting 49 structural metrics at method and class levels. We performed a correlation analysis between test smells and structural metrics of the test code. Therefore, we could discuss the correlations regarding the design problems associated with test smells and the properties of the test code. As a result, the analysis of the `TSSM` dataset can improve the precision of correlation analysis between test smells and structural metrics of test code. Our `TSSM` dataset can support future research on the relationship between test smells and production classes.

3 | RESEARCH METHOD

3.1 | Goal and research questions

Our study investigates the co-occurrences of test smells and their relationship with structural metrics of test code. Therefore, we determine the following research questions (RQs):

- **RQ1: What are the most frequently detected test smells in tests written with JUnit?**
  Rationale: We aim to analyze which test smells frequently occur in test classes and methods written with JUnit. In addition, we hypothesize that different test smells can occur according to the size of test suites.
- **RQ2: Is there a significant relationship between the detected test smells?**
  Rationale: We aim to analyze which test smells co-occur in test classes and methods written in JUnit. In addition, we hypothesize that different test smells can co-occur according to the size of test suites.
- **RQ3: Is there a significant relationship between test smells and structural metrics of test code?**
  Rationale: We aim to investigate the relationship between the test smells and the structural metrics of the test code. In addition, we hypothesize that different test smells and structural metrics of test code can co-occur according to the size of test suites.

Figure 1 shows an overview of the employed research method in our research. We executed three steps through automated support: (1) *Mining GitHub*: to select the subject systems; (2) *Extracting data*: to calculate the structural metrics of test code, detect test smells and establish a relationship between both; and (3) *Clustering projects*: to organize the projects into three clusters according to the size of test code. After clustering, we performed the statistical analysis to answer the RQs.

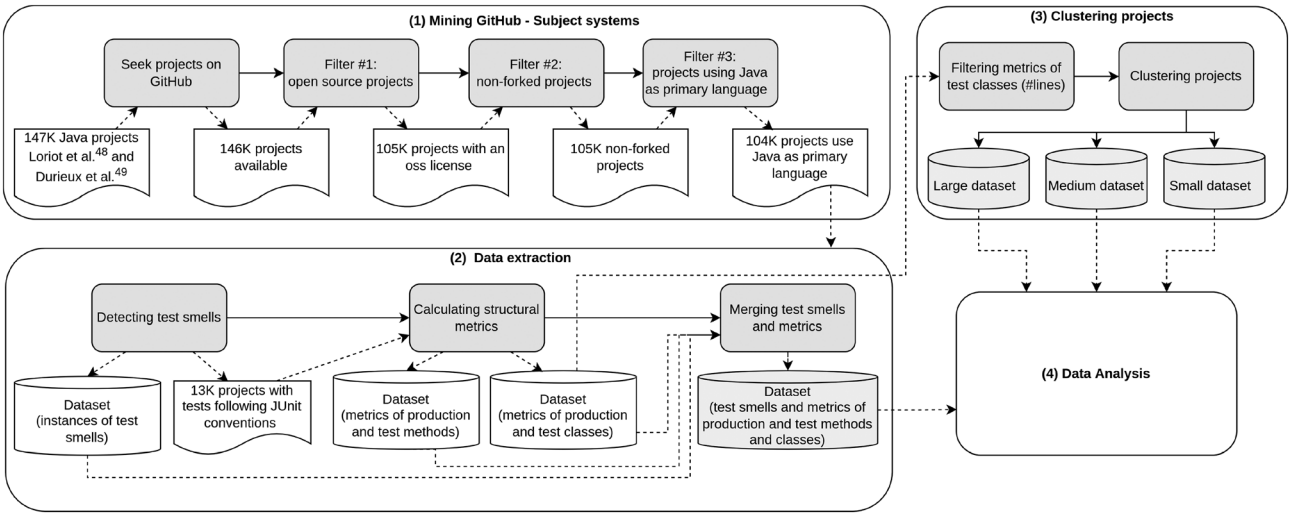


FIGURE 1 Mining open-source Java projects from GitHub

3.2 | Subject systems

Mining the GitHub repository is time-consuming because it has more than 8 million projects. Therefore, we used the GHRepository[††] to seek 147,991 projects listed by Lori et al[50] and Durieux et al.[51] We collected the metadata from each project: id, name, owner, date of creation and last update, number of stars, subscribers, forks, issues, license, size, and last commit. Subsequently, we applied the selection criteria:

1. **Filter #1: open-source projects.** Fogel et al[52] stated that the repository should make it unambiguously clear that the project is open-source. Such an omission might lose many potential developers and users. In addition, several ethics issues can arise when mining software repositories (e.g., consent to study the available data).[53] Therefore, we limited our research to projects with a declared license compliant with OSI (Open-Source Initiative) or FSF (Free Software Foundation) licensing as a strategy to filter out toy and learning-based projects;
2. **Filter #2: non-forked projects.** Jarczyk et al[54] stated that forks are often created to support the development process and do not necessarily constitute separate projects. Therefore, we removed the forked projects because they contain excerpts of code similar to the original ones, which return similar values in the data extraction, biasing the results;
3. **Filter #3: projects using Java as a primary language.** The initial list contains projects that use Java as the primary programming language. As the projects have evolved since the GitHub mining, we analyzed whether they continue using Java as a primary language.

3.3 | Data extraction

The `TSSM` dataset contains data from 13,703 open-source Java projects. We used different tools available in the literature to collect the data, the `JNose Test` tool to detect test smells, and the `CK Metrics` tool to calculate the structural metrics of test code and production code. Then, we merged the data to compose our dataset. More specifically, we performed the steps:

- **Detecting test smells.** Our study focused on problems related to test code quality, especially regarding the presence of test smells. In a systematic mapping study, Aljedaani et al[42] identified 22 tools for detecting test smells. Among the detection tools for Java code, `JNose Test` and `tsDetect` stand out for detecting 19 and 21 test smells, respectively. Both tools perform a test code static analysis through an AST (Abstract Syntax Tree) to apply the test smells detection rules. The `tsDetect` tool presents a precision score ranging from 85% to 100% and a recall score ranging from 90% to 100%. The `JNose Test` tool was built based on the `tsDetect` tool, making the execution process fully automated through an API (Application Programming Interface) and keeping a precision score ranging from 85% to 100% and a recall score from 90% to 100%[18,23] in coarse granularity (class level). In addition, the `JNose Test` tool detects test smells in fine granularity (method, block, and line levels) with a precision score ranging from 84% to 100% and a recall score from 47% to 100%. More specifically, that tool generates `.csv` files with the location of test smells in fine granularity, which we used to add a value of zero or one for a given test smell at method level. Then, the tool sums up the test smells to a coarse granularity. Therefore, we chose the `JNose Test` tool to identify whether the projects have test files written with JUnit and follow the naming conventions (i.e., pre-pending or appending the word "Test" to the name of the production class under test and at the same package hierarchy).[19] We selected 19 test smells with proven precision and recall scores to compose a diverse catalog of design problems related to different characteristics of test code in different granularities (Table 1).
- **Calculating structural metrics.** We collected data from 49 structural metrics, where 28 metrics at class level (Table 2) and 21 metrics at method level (Table 3) to express the test and production code complexity, coupling, cohesion, and size.[55] We used the `CK metrics` tool, which performs a static analysis of the test and production classes code through an AST to calculate the metrics.[24] As a result, the tool returns two `.csv` files (one file with the metrics values at class level and another with the metrics values at method level).
- **Merging test smells and metrics.** We analyzed the data at method and class levels to establish traceability links between the `JNose Test` and `CK Metrics` tools. We followed the JUnit naming conventions of either pre-pending or appending the word `Test` to the name of the production class or method at same level of the package hierarchy. For example, a production class in the package `/src/java/example/` is called `ExampleName.java`, so its test class should be in the package `/src/test/example` and named `ExampleNameTest.java` or `Test-ExampleName.java`. Some limitations of using the naming conventions to match production and test classes include: i) no all production classes of a project match with their respective test classes, ii) not all production methods match with their respective test methods, iii) test smells that occur across test methods (i.e., the *Lazy Test* test smell) require a different approach to match them with structural metrics. Therefore, we considered a proportion of one production class/method to one test class/method, and we did not analyze the *Lazy Test* test smell in this study.

3.4 | Clustering projects

Zhou et al[56] reported the role of size as a possible confounding effect when studying the change-proneness of code elements. Following this statement, Spadini et al[4] found out that the likelihood of a test case being smelly and more change- or defect-prone varies when controlling for

TABLE 1 Description of test smells as detected by JNose Test[23]

Acronym	Test Smell	Description	Level	Precision	Recall	Design Problems
AR	Assertion Roulette	An assertion statement that does not contain an explanation	Line	100%	100%	Test semantic/logic
CI	Constructor Initialization	A test class that contains a constructor instead of a setup method	Method	100%	100%	Issues in test steps
CTL	Conditional Test Logic	A test method that contains conditional and loop statements	Block	100%	100%	Test semantic/logic
DA	Duplicate Assert	Assertion statements in a method contains the same parameters	Line	100%	94%	Code Related
ECT	Handling Exception	A test method that contains throws statements	Block	100%	100%	Issues in Test Steps
EpT	Empty Test	A test method that does not contain executable statements	Method	100%	100%	Issues in Test Steps
ET	Eager Test	A test method contains multiple calls to multiple class under test methods	Line	100%	89%	Test semantic/logic
GF	General Fixture	Fields within the setUp method are not utilized by all test methods	Method	100%	90%	Issues in test steps
IgT	Ignored Test	A test method that contains an annotation to ignore the method	Method	100%	100%	Test Execution
LT	Lazy Test	Multiple test methods call the same class under test methods	Method	100%	97%	Test semantic/logic
MG	Mystery Guest	A test method using instances of external files and databases	Method	100%	50%	Dependencies
MNT	Magic Number Test	An assertion that contains numbers as a parameter	Line	100%	95%	Code Related
PS	Print Statement	A statement invokes print methods	Line	100%	100%	Code Related
RA	Redundant Assertion	An assertion with the same expected and actual parameters	Line	100%	100%	Issues in Test Steps
RO	Resource Optimism	A test method that does not verify the existence of a file before using it	Method	84%	47%	Dependencies
SE	Sensitive Equality	A statement that invokes a method to convert the parameters into strings	Line	100%	100%	Code Related
ST	Sleepy Test	A statement invokes a sleep thread	Line	100%	100%	Test Execution
UT	Unknown Test	A test method that contains a body without assertions	Method	100%	100%	Issues in Test Steps
VT	Verbose Test	A test method that contains more than 30 lines	Method	100%	100%	Code Related

size and number of changes. Similarly, Chowdhury et al[57] found out that the length of the method influences the maintenance effort as a group of small methods is generally less change- and bug-prone than a group of large methods. Thus, we hypothesize that the diffusion and co-occurrences of test smell and the relationship between test smells and structural metrics can vary according to the test suite size.

As pointed out by Kochhar et al,[58] projects with test cases are bigger in terms of LOC than projects without them; however, the number of tests per LOC decreases as projects get bigger and more complex. Therefore, we used the number of test code lines (LOC-Test) to group projects into 70th, 80th, and 90th percentiles.[4] The small cluster has LOC-Test $\leq$ 1468, the medium size cluster has 1468 < LOC-Test $\leq$ 6993, and the large cluster has LOC-Test > 6993.

3.5 | Data analysis

Although our dataset contains metrics values of both production and test code, we only considered the metrics from the test code to perform the replication study. We selected a set of 18 metrics out of the 49 metrics in our dataset because they are a combination of other metrics. For

TABLE 2 Description of structural metrics at class level

Acronym	Metric	Description	Property
Metrics composing the dataset and analyzed in the replication study			
CBO	Coupling between Objects	Counts the number of dependencies a class has	Coupling
DIT	Depth of Inheritance Hierarchy	Counts the number of "fathers" a class has	Inheritance
LCOM	Lack of Cohesion of Methods	Calculates a normalized metric for the lack of cohesion	Cohesion
NOF	Number of fields	Counts the number of fields.	Size
NOM	Number of Methods	Counts the number of methods	Size
RFC	Response for a Class	Counts the number of method invocations in a class	Complexity
WMC	Weighted Methods Complexity	Counts the number of branch instructions in a class	Complexity
Other metrics composing the dataset			
abstractMethodsQty	Number of public abstract methods	Counts the number of public abstract methods	Size
anonymousClassesQty	Quantity of Anonymous classes	Counts the number of anonymous classes	Size
defaultFieldsQty	Number of default fields	Counts the number of public default fields	Size
defaultMethodsQty	Number of public default methods	Counts the number of public default methods	Size
finalFieldsQty	Number of final fields	Counts the number of public final fields	Size
finalMethodsQty	Number of public final methods	Counts the number of public final methods	Size
innerClassesQty	Quantity of inner classes	Counts the number of inner classes	Size
LCC	Loose Class Cohesion	Normalizes the number of (in)direct connections	Cohesion
privateFieldsQty	Number of private field	Counts the number of private fields	Size
privateMethodsQty	Number of private methods	Counts the number of private methods	Size
protectedFieldsQty	Number of protected fields	Counts the number of public protected fields	Size
protectedMethodsQty	Number of public protected methods	Counts the number of public protected methods	Size
publicFieldsQty	Number of public fields	Counts the number of public fields	Size
publicMethodsQty	Number of public methods	Counts the number of public methods	Size
staticFieldsQty	Number of static fields	Counts the number of static fields	Size
staticInvocationQty	Number of static invocations	Counts the number of invocations to static methods	Size
staticMethodsQty	Number of static methods	Counts the number of static methods	Size
synchronizedFieldsQty	Number of synchronized fields	Counts the number of public synchronized fields	Size
synchronizedMethodsQty	Number of public synchronized methods	Counts the number of public synchronized methods	Size
TCC	Tight Class Cohesion	Normalizes the number of direct connections	Cohesion
visibleMethodsQty	Number of public visible methods	Counts the number of public visible methods	Size

example, the *Number of Methods* metric indicates the number of methods in a class, which includes the values of the *staticMethodsQty*, *publicMethodsQty*, *privateMethodsQty*, *protectedMethodsQty*, *defaultMethodsQty*, *visibleMethodsQty*, *abstractMethodsQty*, *finalMethodsQty*, *synchronizedMethodsQty* metrics.

In addition, we can use some metrics calculated at method level to calculate them at class level. For example, the *methodsInvokedQty* metric counts the number of methods invoked in a method and the number of methods invoked in a class, that is, that metric sums all values of the methods of a class. As the analysis at class level includes data from the method level, we considered a disjoint set of seven class metrics, ten method metrics, and the *Lines of Code* at both granularities.

Then, we analyzed the RQs. In RQ1, we investigated the presence of test smells in the test methods and classes. In RQ2, we investigated the co-occurrence of test smells at method and class levels, that is, how often the presence of one specific test smell implies the presence of another test smell. In the co-occurrence analysis, we measured for each test smell (T_i) the percentage of times that its presence co-occurs with each other test smell (T_j, where $i \neq j$). Therefore, for each pair of test smells (T_i,T_j), we measured that percentage by the equation[15]:

$$co-occurrences_{T_{ij}} = \frac{|T_i \wedge T_j|}{|T_i|}$$

where $|Ti \wedge Tj|$ is the number of co-occurrences of T_i and T_j and $|T_i|$ is the number of occurrences of T_i. Note that co-occurrences T_{ij} differs from co-occurrences $T_{j,i}$ because the denominator changes from $|T_i|$ to $|T_j|$.

TABLE 3 Description of structural metrics at method level

Acronym	Metric	Description	Property
Metrics composing the dataset and analyzed in the replication study			
LOC	Lines of Code	Counts the lines, ignoring empty lines and comments	Size
loopQty	Quantity of loops	Counts the number of loops (i.e., for, while, do while)	Size
maxNestedBlocksQty	Max nested blocks	The highest number of blocks nested together	Size
methodsInvokedQty	Quantity of method invocations	Counts the number of output dependencies	Coupling
numbersQty	Quantity of Numbers	Counts the number of numeric literals	Size
parametersQty	Quantity of parameters	Counts the number of parameters of a method	Size
returnQty	Quantity of returns	Counts the number of return instructions	Size
stringLiteralsQty	Quantity of string literals	Counts the number of string literals (e.g., "John Doe")	Size
tryCatchQty	Quantity of try/catches	Counts the number of try/catches	Size
uniqueWordsQty	Number of unique words	Counts the number of unique words	Size
variablesQty	Quantity of Variables	Counts the number of declared variables	Size
Other metrics composing the dataset			
assignmentsQty	Qty of assignments	Counts the number of assignments	Size
comparisonsQty	Quantity of comparisons	Counts the number of comparisons (i.e., == and !=)	Size
hasDoc	Has Javadoc	Boolean indicating whether a method has javadoc	Size
lambdasQty	Quantity of lambda expressions	Counts the number of lambda expressions	Size
logStatementsQty	Quantity of Log Statements	Counts the number of log statements	Size
mathOperationsQty	Quantity of Math Operations	Counts the number of math operations (times, divide, remainder, plus, minus, left shit, right shift)	Size
methodsInvIndLocQty	Quantity of local methods invoked indirectly	Counts the local methods invoked indirectly	Coupling
methodsInvLocQty	Quantity of local methods invoked directly	Counts the number of local methods invoked directly	Coupling
modifiers	Quantity of modifiers	Counts the number of public/abstract/private/protected/native modifiers of classes/methods	Size
parenthesizedExpsQty	Quantity of parenthesized expressions	Counts the number of expressions inside parenthesis	Size

In RQ3, we investigated the correlation between test smells and structural metrics of test code at class and method levels. Based on the different data distributions in our dataset, we chose the Spearman Rank Test non-parametric test to investigate that correlation, considering a significance level of 5%. The Spearman Rank Test returns a coefficient (ρ) that ranges from -1 to $+1$ to measure how two variables are associated with a monotonic function—a value of ρ close to zero implies both variables are not correlated. Otherwise, the variables are correlated and have an increasing or decreasing relationship[59] that we interpreted following the intervals proposed by Salking and Rainwater.[60] Therefore, we assumed no correlation when $0.00 \leq \rho \leq 0.20$, weak correlation when $0.21 \leq \rho \leq 0.40$, moderate correlation when $0.41 \leq \rho \leq 0.60$, strong correlation when $0.61 \leq \rho \leq 0.80$, and very strong correlation when $0.81 \leq \rho \leq 1.00$. Similar intervals also apply to negative correlations.

In addition, we investigated the RQs considering three clusters to identify whether the test suites of different sizes present a different outcome. Given the skewed distribution of our dataset, we chose the Kruskal–Wallis test[61] to verify whether there is a significant difference in the distribution of test smells among the clusters, with a significance level of 5%. The Kruskal-Wallis Test returns a $p-value$, which indicates whether the groups' medians are (not) equal. If the $p-value$ is less than or equal to the significance level, the group's medians are not equal.

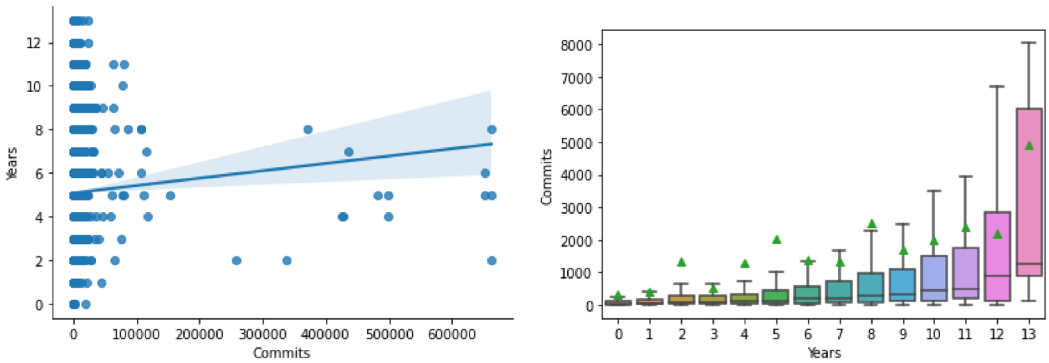
4 | RESULTS AND DISCUSSION

4.1 | Projects characterization

Through the GitHub mining step (Figure 1), we gathered 13,703 subject systems to compose our dataset. Table 4 presents the descriptive statistics of our dataset in terms of average (*Mean*), standard deviation (*SD*), minimum value (*Min*), first quartile (*Q1*), second quartile or median (*Median*), third quartile (*Q3*), maximum value (*Max*) and the sum of values (*Total*). Most projects have up to 77 stars and 35 forks (Q3: 75% of the number of

TABLE 4 TSSM Dataset Characterization

	Mean	SD	Min	Q1	Median	Q3	Max	Total
Stars	238.3	1478.0	0	10	23	77	70,893	3,264,731
Forks	82.0	587.6	0	4	11	35	44,923	1,123,478
#Test-Class	77.7	331.6	1	4	13	43	9940	1,064,227
#Prod-Class	376.0	1983.2	1	25	77	245	149,108	5,152,360
#Test-Method	383.2	1614.8	1	16	55	200	44,372	5,251,462
#Prod-Method	2469.3	14,233.7	1	110	372	1271	683,666	33,836,748
#LOC-Test	4161.3	18,832.1	4	142	525	1985	757,087	57,021,856
#LOC-Prod	22,231.4	114,274.1	5	880	3009	10,732	5,037,600	304,637,181



(A) Correlation between the Characterization of the lifetime of the projects and number of commits

(B) Distribution of commits per lifetime of the projects (years)

FIGURE 2 Characterization of the lifetime of the projects (years) and number of commits

Stars and Forks). Based on the metrics calculated by the CK Metrics tool (i.e., considering enum, inner, interface, and anonymous classes), we analyzed 361,659,037 lines of code (LOC), where 57,021,856 lines of test code (LOC-Test) and 304,637,181 lines of production code (LOC-Prod). These are distributed in 6,216,587 classes, where 1,064,227 test classes (#Test-Class) and 5,152,360 production classes (#Prod-Class). Thus, the TSSM dataset contains five times more data related to production classes than test classes.

In addition, Figure 2a presents the longevity (years) and the number of commits (until June 2022) of the projects from the TSSM dataset in a scatterplot. The lifetime of the projects and the number of commits have a weak positive correlation (0.278). Figure 2b groups the projects by lifetime and presents their number of commits. The median number of commits increases as the project lifetime increases. The group of projects with 13 years is the smallest one (0.1%) and has some particularities. That group has a minimum of 108 commits, a maximum of 23,988, a mean of 4922.58, and a standard deviation of 7365.63 commits. Thirteen-year projects present a low deviation of values compared to those between 4 and 8 years. For example, 8-year projects have a minimum of 1 commit, a maximum of 662,227, a mean of 2517.12, and a standard deviation of 24,526.14. Still, most projects (61%) have a lifetime between four and eight years.

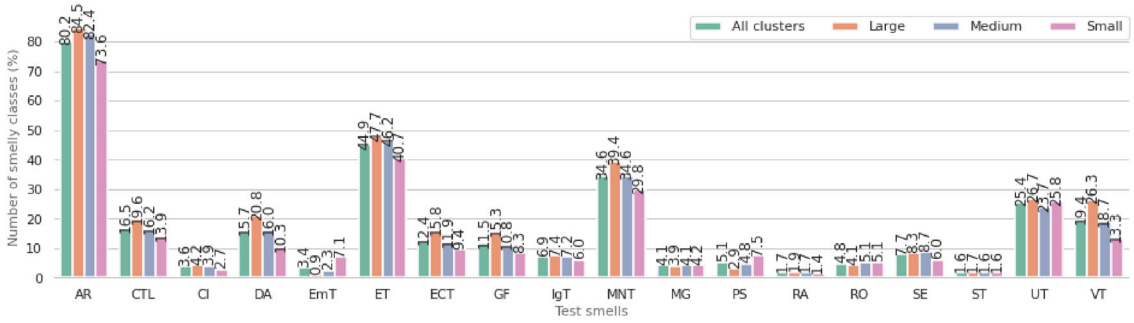
Next, we clustered the projects to analyze the RQs (Section 3.4). We separated the projects based on the number of LOC-Test, grouping the projects of the 70th, 80th and 90th percentiles in the small, medium, and large clusters.[4] The small cluster has 9588 projects, the medium cluster has 2743 projects, and the large cluster has 1372 projects.

4.2 | Distribution and frequency of test smells (RQ1)

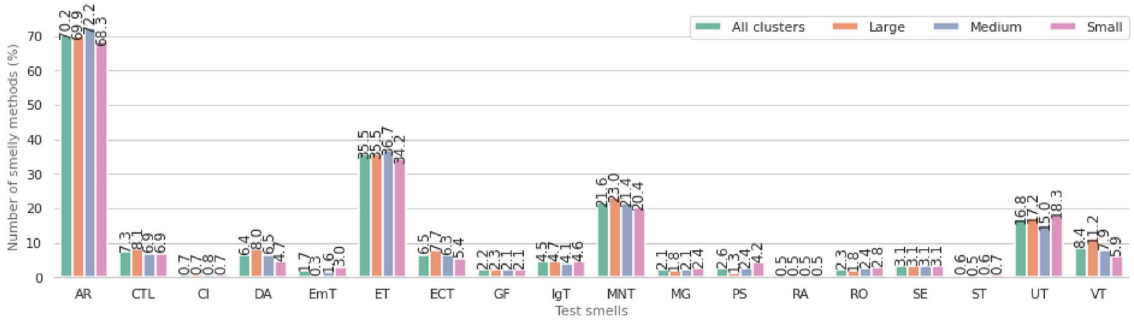
Table 5 describes the clusters in terms of the number of projects (#Projects), number of test classes (#Test-Class), number of production classes (#Prod-Class), number of test methods (#Test-Method), number of production methods (#Prod-Method), number of lines in test classes (#LOC-Test), number of lines in production classes (#LOC-Prod), number of smelly test classes (#Smelly-Class), number of smelly test methods (#Smelly-Method), and number of test smells instances (#Smells). It is worth mentioning that hereafter we do not consider the enum, inner, interface, and anonymous classes to match the JNose Test and CK Metrics outputs. In comparison with the description of the entire dataset in Table 4, the

TABLE 5 Characterization of smelly test classes in the `TSSM` dataset and the clusters after merging the `JNose Test` and `CK Metrics` tools' outputs

	#Projects	#Test-Class	#Prod-Class	#Test-Method	#Prod-Method	#LOC-Test	#LOC-Prod	#Smelly-Class	#Smelly-Method	#Smells
Small	9588	82,394	676,775	374,104	4,753,969	3,468,810	47,436,664	46,204	184,101	346,634
Medium	2743	128,599	641,811	780,963	5,050,692	8,065,916	51,648,900	66,451	345,147	667,240
Large	1372	411,185	1,582,012	3,316,357	16,162,373	40,210,046	153,115,263	174,683	1,126,529	2,225,759
Total	13703	622,178	2,900,598	4,471,424	25,967,034	51,744,772	252,200,827	287,338	1,655,777	3,239,633



(A) Number of smelly test classes.



(B) Number of smelly test methods.

FIGURE 3 Presence of test smells in the test classes and test methods grouped by the dataset size

dataset remains with 58.5% of the test classes (#Test-Class), 85.1% of test methods (#Test-Method), and 90.7% lines of test code (LOC-Test) after merging the tools' outputs.

The `TSSM` dataset has 287,338 smelly test classes, that is, 46.2% of the matched test classes have test smells. Despite the small cluster having more projects than the other clusters, the number of smelly test classes, smelly test methods, and occurrences of test smell increase according to the size of the cluster. From a total of 287,338 smelly test classes, the large cluster contains 174,683 (60.8%) smelly test classes, the medium cluster contains 66,451 (23.1%) smelly test classes, and the small cluster contains 46,204 (16.1%) smelly test classes. Out of 1,655,777 smelly test methods, the large cluster has 1,126,529 (68.0%) smelly test methods, the medium cluster has 345,147 (20.8%) smelly test methods, and the small cluster has 184,101 (11.2%) smelly test methods. Out of 3,239,633 occurrences of test smells, the large cluster has 2,225,759 (68.7%) occurrences of test smells, the medium cluster has 667,240 (20.6%) occurrences of test smells, and the small cluster has 346,634 (10.7%) occurrences of test smells.

Figure 3 shows the percentage of test smells grouped by clusters at test class (Figure 3a) and test method (Figure 3b) levels, indicating one test class or method can have various test smells. Figure 3a shows the three clusters have a similar ratio of test smells at class level. The three most frequent test smells considering the entire dataset are *Assertion Roulette* present in 80.2% of the smelly test classes, *Eager Test* present in 44.9% of the smelly test classes, and *Magic Number Test* present in 34.6% of the smelly test classes. The *Assertion Roulette* test smell is present in 84.5%, 82.4%, and 73.6% of the test classes in the large, medium, and small clusters, respectively. Next, the *Eager Test* test smell is present in 47.7%, 46.2%, and 40.7%, and the *Magic Number Test* test smell is present in 39.4%, 34.6%, and 29.8% of the test classes in the large, medium,

and small clusters, respectively. In contrast, the *Sleepy Test* and *Redundant Assertion* test smells are rare in the three clusters. The *Sleepy Test* test smell is present in 1.7%, 1.6%, and 1.6% of the test classes, and the *Redundant Assertion* test smell is present in 1.9%, 1.7%, and 1.4% of test classes in the large, medium, and small clusters, respectively.

Figure 3b shows a similar ratio of smelly test methods in the three clusters. We can notice a similar pattern of the most common and rare test smells in the analysis at class and method levels. The most frequent test smells are the *Assertion Roulette* test smell in 69.9%, 72.2%, and 68.3%, the *Eager Test* test smell in 35.5%, 36.7%, and 34.2%, and the *Magic Number Test* test smell in 23.0%, 21.4%, and 20.4% of test methods in the large, medium, and small clusters, respectively. In contrast, the *Sleepy Test* (0.5%, 0.6%, and 0.7% in the large, medium, and small clusters, respectively) and *Redundant Assertion* (0.5% in all clusters) test smells are rare in the three clusters at method level. Although the small cluster has a low percentage of most test smells than the other clusters, the percentage of test classes and methods with the *Empty Test* and *Print Statement* test smells is higher in the small cluster than the other clusters, and the *Unknown Test* test smell is higher in the small cluster than the medium cluster. The last three test smells are similar because the test code does not have assertions, that is, it always passes.

In addition to analyzing the presence of test smells, we applied the Kruskal–Wallis test[61] to verify whether there is a significant difference among the diffusion of test smells in the clusters at class level. The dependent variable is the occurrence of test smells (organized into 18 test smells at class level), and the independent variable is the size of test suites. There is a significant difference between the three clusters regarding the number of occurrences of most test smells at class level, except for the *Sleepy Test* test smell ($p\text{-value} = 0.17$). In addition, there is no significant difference between pairs of clusters for the *Resource Optimism* test smell (small and medium clusters, with $p\text{-value} = 0.96$), the *Mystery Guest* test smell (small and medium, with $p\text{-value} = 0.32$, medium and large clusters, with $p\text{-value} = 0.07$), and the *Ignored test* test smell (medium and large, with $p\text{-value} = 0.13$). In addition, those four test smells rarely occur in the test classes, indicating they can be more severe than the others.

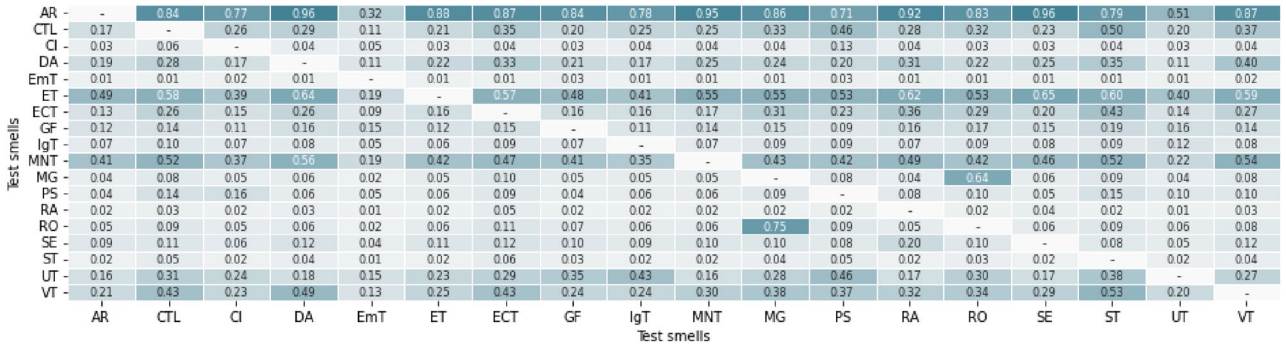
Finding 1. Our dataset contains 46.2% of smelly test classes. Most of the smelly test classes and methods contain the *Assertion Roulette*, *Eager Test*, and *Magic Number Test* test smells. There is a significant difference between the occurrences of test smells at class level in all three clusters, except for the *Sleepy Test*, *Resource Optimism*, *Mystery Guest*, and *Ignored Test* test smells. In addition, those test smells rarely occur in the test classes, indicating that they are more severe than the others.

Discussion. An unexpected result was only 46.2% of test classes from 13,703 subject systems considered in this study are affected by at least one test smell. Bavota et al.,[15] Bavota et al,[11] and Virgínio et al[16] reported that at least 80% of the test classes have test smells. However, they mostly analyzed projects from the Apache Foundation. Differently, Pecorelli et al[26] analyzed software projects from different ecosystems and organizations and found only 59% of the projects have a test suite. We believe our results occurred due to two factors: (i) we considered projects from different ecosystems, and (ii) the analysis depends on the projects following the JUnit naming conventions, that is, the `JNose Test` tool can miss some smelly test classes. The *Assertion Roulette*, *Eager Test*, and *Magic Number Test* test smells are highly distributed in the `TSSM` dataset at class and method levels in the three clusters. In contrast, the *Mystery Guest*, *Resource Optimism*, and *Sleepy Test* test smells have low distribution, converging with Spadini et al.'s[10] findings. On the one hand, test smells highly distributed in the test classes can have a low impact on the test code maintainability. On the other hand, rare test smells are severe and can have a high impact on test code maintainability. Therefore, understanding whether the test smells represent an actual problem for software maintenance is significant. The distribution of test smells also confirms Virgínio et al.'s[16] findings and, partially, Bavota et al.'s,[15] Bavota et al.'s,[11] and Palomba et al.'s[5] findings, except for the *Sleepy Test* test smell.

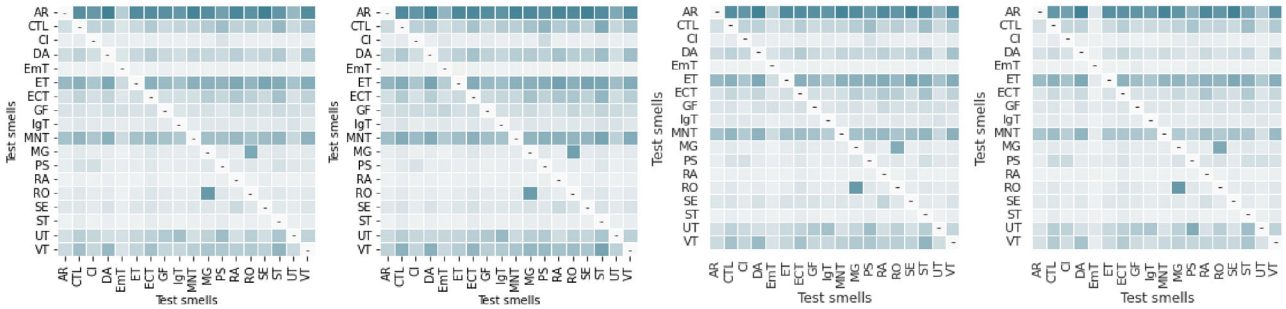
4.3 | Co-occurrence of test smells (RQ2)

In the RQ2, we investigated to what extent test smells co-occur at class and method levels. We calculated the co-occurrences for all pair-wise combinations of test smells (18×18), that is, we analyzed whether the presence of a test smell (T_i) implies the presence of another test smell (T_j) in the same test class or the same test method.

As a result, the three clusters presented similar strong to very strong co-occurrences at class level and similar co-occurrences at method level. Therefore, we presented the co-occurrences in each cluster and discussed them considering the entire dataset. Figure 4a presents the co-occurrence of test smells at class level, and Figures 4b to 4e present the comparison among clusters. The *Mystery Guest* and *Resource Optimism* test smells co-occur (0.75) at the lower triangle of the matrix, and the *Resource Optimism* and *Mystery Guest* test smells co-occur (0.64) at the upper triangle of the matrix. It indicates that both test smells co-occur in most test classes. On the one hand, the *Verbose Test* (0.87), *Sleepy Test* (0.79), *Sensitive Equality* (0.96), *Resource Optimism* (0.83), *Redundant Assertion* (0.92), *Print Statement* (0.71), *Mystery Guest* (0.86), *Magic Number Test* (0.95), *Ignored Test* (0.78), *General Fixture* (0.84), *Exception Catching Throwing* (0.87), *Eager Test* (0.88), *Duplicate Assert* (0.96), *Constructor*

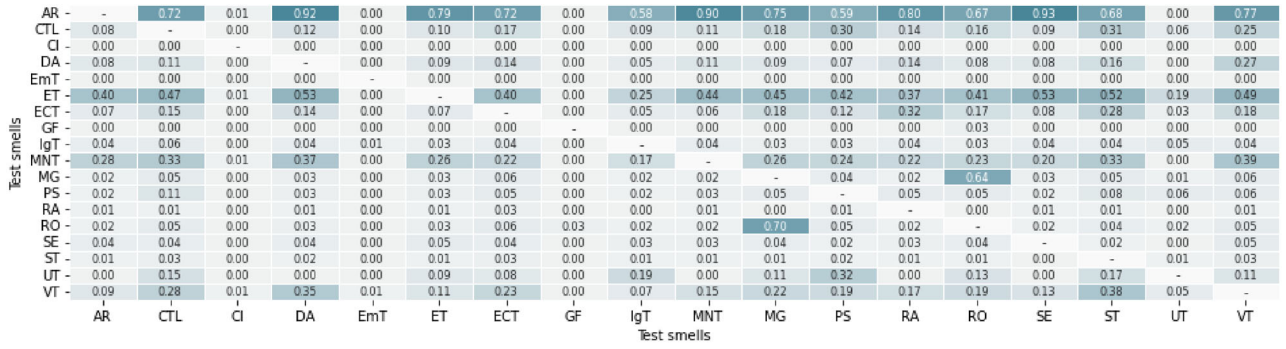


(A) Co-occurrences in all three clusters

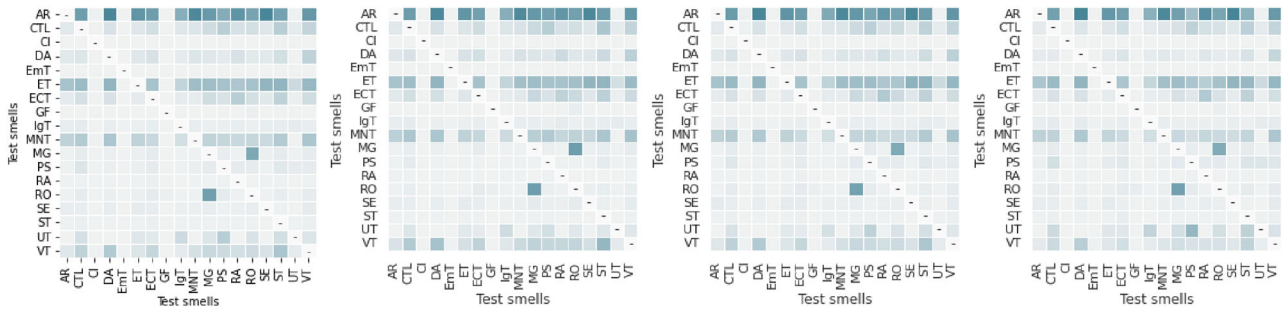


(B) All three clusters **(C)** Large cluster **(D)** Medium cluster **(E)** Small cluster

FIGURE 4 Co-occurrences among test smells at class level



(A) Co-occurrences in all three clusters



(B) All three clusters **(C)** Large cluster **(D)** Medium cluster **(E)** Small cluster

FIGURE 5 Co-occurrences among test smells at method level

Initialization (0.77), *Conditional Test Logic* (0.84) test smells co-occur with the *Assertion Roulette* test smell at the upper triangle of the matrix. On the other hand, the *Assertion Roulette* test smell does not co-occur with the priors test smells in the lower triangle of the matrix. It indicates the test suites have a high diffusion of the *Assertion Roulette* test smell, leading other test smells to co-occur with it.

Figure 5a presents the co-occurrences of test smells at method level in the entire dataset, and Figures 5b to 5e present the comparison among clusters. Similar to the class level, the *Mystery Guest* and *Resource Optimism* test smells co-occur (0.70) at the lower triangle of the matrix, and the *Resource Optimism* and *Mystery Guest* test smells co-occur (0.64) test smells co-occur at the upper triangle of the matrix. The co-occurrences at class level among *Unknown Test* (0.00), *General Fixture* (0.00), *Empty Test* (0.00), and *Constructor Initialization* (0.01) test smells with the *Assertion Roulette* test smell at the upper triangle of the matrix no longer exists at method level. The *Verbose Test* (0.77), *Sleepy Test* (0.68), *Sensitive Equality* (0.93), *Resource Optimism* (0.67), *Redundant Assertion* (0.80), *Mystery Guest* (0.75), *Magic Number Test* (0.90), *Exception Catching Throwing* (0.72), *Eager Test* (0.79), *Duplicate Assert* (0.92),*Conditional Test Logic* (0.72) test smells co-occur with the *Assertion Roulette* test smell at the upper triangle of the matrix. Those co-occurrences are slightly weak at method level.

Regarding the strong co-occurrence at method and class levels, the *Mystery Guest* and *Resource Optimism* test smells refer to the usage of external resources. Although the *Mystery Guest* test smell occurs when a test method contains object instances of files and database classes, the *Resource Optimism* test smell refers to using such files and databases without checking their existence. Listing 2 presents an excerpt of the `SosMetadataUpdateTest` class of the `SensorWebClient` project.[‡‡] The `testGetCacheTarget()` method uses the path indicated at the `cacheTargetFile` attribute to instantiate the `expected` file instead of mocking it, which corresponds to a *Mystery Guest* test smell. Subsequently, the `getAbsoluteFile()` method is called on the `expected` file without verifying if the file exists, which corresponds to a *Resource Optimism* test smell.

```
public class SosMetadataUpdateTest {

    private String cacheTargetFile;

    @Test
    public void testGetCacheTarget() {
        File expected = new File(cacheTargetFile);
        File cacheTarget = SosMetadataUpdate.getCacheTarget(validServiceUrl);
        assertEquals(expected.getAbsoluteFile(), cacheTarget.getAbsoluteFile());
    }
}
```

Listing 2: An example of the detection of the Mystery Guest and Resource Optimism test smells at class and method levels

In addition, we noticed other interesting co-occurrences at method level. The *Constructor Initialization* test smell does not co-occur with other test smells because the constructor method is untagged with `@Test`, `@Before`, `@After`, etc. Listing 3 presents the `ContainerEventSourceTest` test class of the `RxSwing` project,[§§] which uses a constructor to instantiate the attributes of the test class instead of one setup method.

```
public class ContainerEventSourceTest {

    private final Func1<Container, Observable<ContainerEvent>> observableFactory;

    public ContainerEventSourceTest(Func1<Container, Observable<ContainerEvent>> observableFactory) {
        this.observableFactory = observableFactory;
    }
}
```

Listing 3: An example of the detection of the Constructor Initialization test smell at method level

The *Empty Test* test smell also does not co-occur with other test smells because its definition refers to test methods without executable statements. Similarly, the *Unknown Test* test smell has executable statements but no assertions. Thus, the *Unknown Test* test smell can co-occur with the *Print Statement* test smell but does not co-occur with the *Assertion Roulette*, *Duplicate Assert*, and *Redundant Assertion* test smells. Listing 4 presents the `WxMpPayServiceImplTest` class of the `WxJava-for-JDK6` project.[¶¶] The `testCheckJSSDKCallbackDataSignature()` method does not have executable statements, characterizing one *Empty Test* test smell. The `testSendRedpack()` method uses a print statement to exhibit the test result instead of using an assertion, characterizing one *Unknown Test* and one *Print Statement* test smell.

```
public class WxMpPayServiceImplTest {

    @Test
    public void testCheckJSSDKCallbackDataSignature() throws Exception {

    }

    @Test
    public void testSendRedpack() throws Exception {
        WxSendRedpackRequest request = new WxSendRedpackRequest();

        request
            .setReOpenid(((WxXmlMpInMemoryConfigStorage) this.wxService.getWxMpConfigStorage()).getOpenid());
        WxRedpackResult redpackResult = this.wxService.getPayService().sendRedpack(request);
        System.err.println(redpackResult);
    }
}
```

Listing 4: An example of the detection of the Empty Test, Unknown Test, and Print Statement test smells at method level

Most co-occurrences of test smells emerge within the same design problem (Table 1). The *Test Semantic/Logic* design problem presents a co-occurrence between the *Assertion Roulette* and *Eager Test* test smells, the *Dependencies* design problem presents a co-occurrence between the *Resource Optimism* and *Mystery Guest* test smells, the *Code Related* design problem presents a co-occurrence between the *Duplicate Assert* and *Magic Number Test* test smells. All those design problems co-occur within the *Test Semantic/Logic* design problem. Differently, the test smells in the *Issues in Test Steps* and *Test Execution* design problems do not co-occur within the same design problem.

Finding 2: The *Mystery Guest* and *Resource Optimism* test smells co-occur at class and method levels, referring to the usage of external resources. Other co-occurrences emerge only at class level, such as the *Unknown Test* and *Empty Test* test smells co-occur with the *Assertion Roulette* test smell.

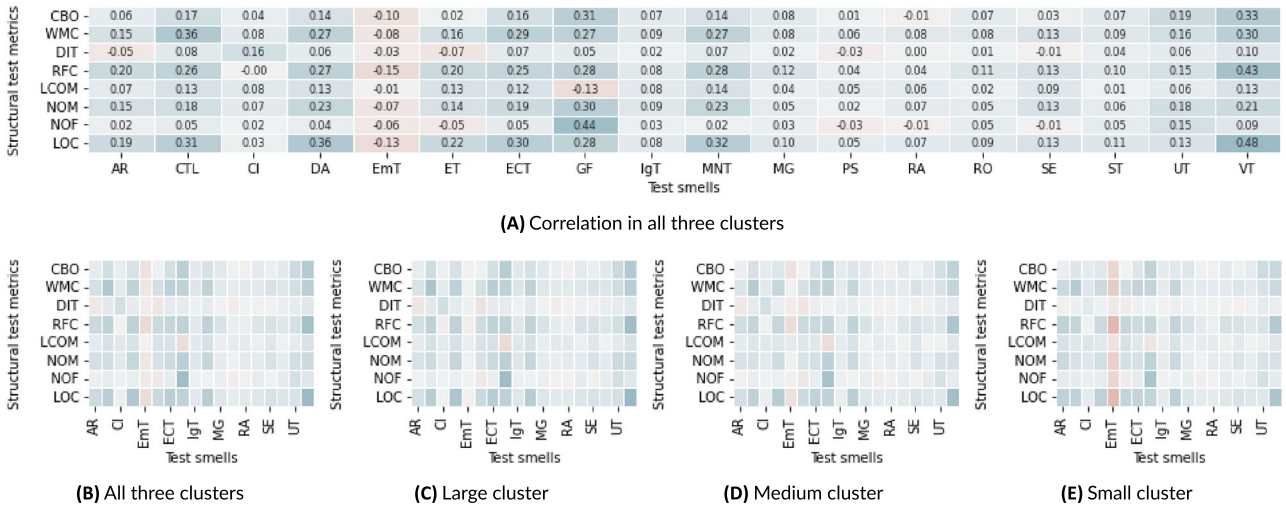
Discussion. We found most test smells frequently co-occur with the *Assertion Roulette*, except for four test smells: (i) *Empty Test*: refers to test methods without executable statement, (ii) *Unknown Test*: refers to test methods without assertions, (iii) *Constructor Initialization*: initializes objects through a constructor instead of one setup method, and (iv) *General Fixture*: accesses properties of the *setUp* methods, which do not assert conditions of the production classes (i.e., *setUp* methods create common test data for all test methods). We also observed a strong co-occurrence between the *Mystery Guest* and *Resource Optimism* test smells, converging with Palomba et al.'s[5] and Virginio et al.'s[16] findings. Thus, whether a test code uses external resources, it probably verifies its use. In addition, we found other interesting co-occurrences at method level described by previous work. The *Constructor Initialization* test smell does not co-occur with other test smells because it is not a test method, that is, it can not have other test smells. The *Empty Test* test smell also does not co-occur with other test smells because it refers to an empty method and other test smells require the test method to have a body. The *Unknown Test* test smell refers to methods without assertions; thus, it does not co-occur with the *Assertion Roulette*, *Duplicate Assert*, and *Redundant Assertion* test smells because they are present only in the assertion statements.

4.4 | Test smells and structural metrics (RQ3)

To answer RQ3, we analyzed the correlations between test smells and structural metrics of the test classes and test methods. We calculated the Spearman's correlation coefficient test for all pair-wise combinations of 18 test smells and eight structural metrics at class level, and 18 test smells and 11 structural metrics at method level. Calculating Spearman's correlation uses the number of occurrences of a test smell (T_i) and the value of a structural metric (S_i) present in the same test class.

Figure 6a presents the correlations between test smells and structural metrics at class level, and Figures 6b to 6e present the comparison among clusters. The analysis indicates the three clusters presented similar correlations at class and method levels, except for the *Empty Test* test smell. That test smell occurs three times more frequently in the small cluster, resulting in a stronger correlation with the structural metrics than in the other clusters. Most test smells have a weak correlation with structural metrics at class level. Only a few moderate correlations occur between the pairs (metric - test smell): *Response for a Class* - *Verbose Test* (0.43), *Lines of Code (LoC)* - *Verbose Test* (0.48), and *Number of Fields* - *General Fixture* (0.44).

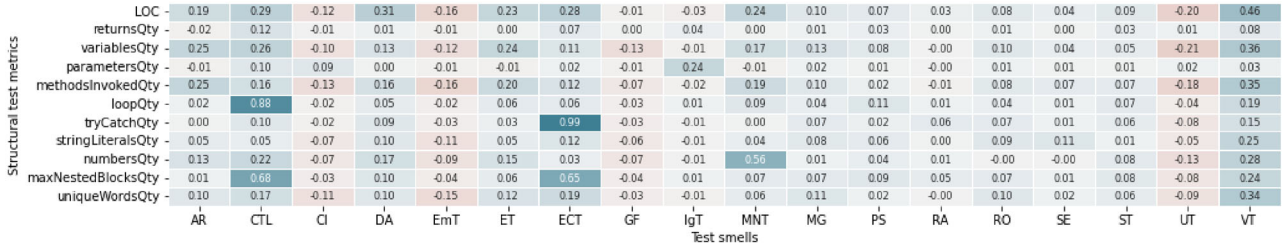
Some test smells have a moderate to very strong correlation with structural metrics at method level. Figure 7a presents the correlations of test smells and structural metrics at method level, considering the entire dataset. Figures 7b to 7e present the comparison between the TSSM



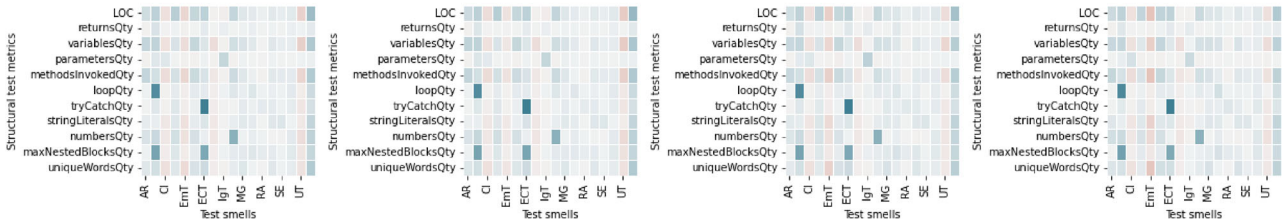
(A) Correlation in all three clusters

(B) All three clusters **(C)** Large cluster **(D)** Medium cluster **(E)** Small cluster

FIGURE 6 Correlations between test smells and structural metrics of test classes



(A) Correlation in all three clusters



(B) All three clusters **(C)** Large cluster **(D)** Medium cluster **(E)** Small cluster

FIGURE 7 Correlations between test smells and structural metrics of test methods

dataset and clusters. The very strong correlations occur between the pairs (metric - test smell): *tryCatchQty* - *Exception Catching Throwing* (0.99) and *loopQty* - *Conditional Test Logic* (0.88). Strong correlations occur between the pairs (metric - test smell): *maxNestedBlocksQty* - *Exception Catching Throwing* (0.65) and *maxNestedBlocksQty* - *Conditional Test Logic* (0.68). Moderate correlations occur between the pairs (metric - test smell): *numbersQty* - *Magic Number Test* (0.56) and *Lines of Code* - *Verbose Test* (0.46).

The correlation analysis indicates that the test smells are mostly related to the complexity, coupling, and size of the test class (Table 2). Thus, cohesion and inheritance are rare in the test classes. In contrast, the test smells relate to a few characteristics regarding the size of the test method. More specifically, the metrics useful for detecting test smells at method level are *maxNestedBlocks*, *numbersQty*, *loopQty*, *tryCatchQty*, and *Lines of Code* (LOC). In addition, the *Empty Test* test smell has a weak negative correlation or no correlation with structural metrics at class level. The same occurs with other test smells in the *Issues in Test Steps* design problem at method level.

Finding 3: Test smells have a moderate correlation with structural metrics, mostly related to the complexity, coupling, and size of the test class. Differently, few structural metrics of the test methods are related to test smells, indicating that those metrics can be useful for detecting test smells.

Discussion. We found that test smells have a weak to moderate correlation with structural metrics related to size (NOM, NOF, LOC), complexity (WMC, RFC), and coupling (CBO). Tahir et al[17] analyzed the co-occurrences of test smells and structural metrics of production code and identified the complexity is a strong indicator of test smells; in particular, for the pairs (test smell - metric): *General Fixture - Lack of Cohesion of Methods* (LCOM), *Eager Test - Cyclomatic Complexity* (CC), *Eager Test - Lack of Cohesion of Methods* (LCOM), and *Eager Test - Weighted Methods Complexity* (WMC). Differently, Bavota et al.,[15] Bavota et al,[11] and Palomba et al[5] identified some strong correlations between the test smells and the size of the system in terms of the number of classes, number of JUnit classes, line of code of the production class, and line of code of the JUnit class. We understand that the divergences in the correlations granularity for the LOC metric (the only metric in common) are due to the analysis of the presence of test smells performed by the authors. Other properties of the production class correlate with test smells. For example, Tufano et al[6] identified the class under test could have code smells correlated with test smells.

5 | THREATS TO VALIDITY

Construct validity. Those threats are related to our research instruments for data extraction. We integrated different APIs to mine the GitHub, calculated the structural metrics, and detected the test smells. We observed failures related to (i) denied permission to get information about the project, (ii) invalid structure of classes (e.g., the `JNose Test` and `CK metrics` tools do not build the ASTs), and (iii) computer not having enough memory for the calculations. Another threat is how we calculated the number of test smells at method level. The `JNose Test` tool detects the test smells in different levels (line, block, method, and class) and generates an output file (`.csv`) containing the test smells. We used the location of smelly lines to indicate whether a test method or test class has one test smell to establish a measurement unit. In addition, the `JNose Test` tool does not distinguish inner test classes from JUnit test classes, considering them as one unique test class, which results in misunderstanding. Therefore, we used the `CK metrics` tool to filter the inner classes from our dataset. Another threat is how we established a traceability link between test smells and structural metrics. We exploited a traceability technique based on naming convention, which can not match all test classes identified by tools. In addition, we considered only JUnit test codes. There could exist projects containing several non-JUnit test classes, affecting the decision to cluster the projects based on the size of the test suite.

External Validity. Those threats are related to the generalization of our results. Our study is limited to open-source Java projects with JUnit test classes due to the requirements of the `JNose Test` tool. However, we analyzed more than 6 million classes from 13,703 open-source Java projects from different domains. Moreover, we clustered the projects according to the size of their test suites, but further investigation is needed to analyze the `TSSM` dataset through different characteristics of the selected projects, such as total size, domain, and maturity.

Conclusion Validity. Those threats are related to the relationship between experimentation and outcomes. We analyzed the presence of test smells in the JUnit test classes and test methods detected by the `JNose Test` tool (RQ1). We used a well-known traceability approach based on naming convention to analyze the co-occurrences of test smells in JUnit test classes and test methods (RQ2). In addition, through appropriate statistical procedures (Spearman Rank Test), we analyzed the correlations of test smells and structural metrics of the test classes and test methods (RQ3). However, a correlation analysis does not imply a cause-effect relationship. Therefore, a qualitative study is crucial to complement it, investigating the reasons behind such correlations.

Replicability. To enhance replicability, we provided a replication pack, including the scripts and results of our data extraction process. However, we collected data using the last version of the projects when we mined GitHub. A threat arises when expanding the `TSSM` dataset with new metrics or project versions. The open-source projects can become unavailable, requiring checking the collected data consistency.

6 | CONCLUDING REMARKS

This paper characterizes a database of 13,703 projects with 1,064,227 test classes regarding test smells and structural metrics. We integrated different APIs to detect test smells (`JNose Test`) and calculate structural metrics of the test code and production code (`CK Metrics`). Both tools use different mechanisms to identify the classes. The `CK Metrics` tool identifies the `.java` classes without differing production to test classes. The `JNose Test` tool identifies the `.java` classes and analyzes the classes' names and their imports to determine whether it is a test class. Results matching those tools indicate at least 46.2% of JUnit classes are affected by at least one test smell. We split the projects into three clusters to analyze the data according to the size of the test suites. Results indicated the *Assertion Roulette* test smell is the most recurring, followed by the *Eager Test* and *Magic Number Test* test smells. Most test smells co-occur with the *Assertion Roulette* test smell due to its high diffuseness, but that test smell only co-occurs with test smells of the *Test semantic/logic* design problem. Another strong co-occurrence emerges between the *Mystery Guest* and *Resource Optimism* test smells at class and method levels related to the *Dependencies* design problem. Therefore, our results confirm and extend those obtained by Bavota et al.,[15] Batova et al.,[11] Palomba et al,[5] and Virgínio et al.[16]

Additionally, our results support and expand those obtained in related works[6,17] that investigated the correlation between test smells and design properties. Tahir et al[17] identified that complexity is a strong indicator of test smells, particularly the *Cyclomatic Complexity* and *Weighted*

Methods per Class metrics. Tufano et al[6] identified that the class under test might have code smells correlated with test smells. When we look at the nature of the code smells, they are mainly related to the coupling and size of the class under test. Although we could identify only moderate correlations between test smells and structural metrics of the test class, our study supports the related work findings. The complexity, coupling, and size properties of the production class correlate with test smells in the corresponding test class. In addition, our study identified moderate to very strong correlations between test smells and structural metrics of test methods, indicating some metrics can be helpful in detecting test smells or be used to calculate the severity of test smells.

It is worthy to notice that we conducted a correlation analysis between test smells and software metrics, which does not imply a cause-effect relationship. The same occurs in the related works. For example, a test class can contain more test smells than others because it is longer. In future work, we intend to set control variables to understand whether the test smells and structural metrics are consistently related and perform a qualitative study to understand the cause-effect relationship. In addition, we could explore other structural metrics and metadata of projects from the `TSSM` dataset to derive a catalog of metrics and severity thresholds helpful to support test smells detection. We could also expand the `TSSM` dataset with data of bug fixes and coverage to investigate whether test smells make the test case ineffective at locating bugs.

ACKNOWLEDGMENTS

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001; and FAPESB grants JCB0060/2016 and BOL0188/2020.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in TSSM Dataset at https://figshare.com/s/eee9374e35d9463cf1ff.

ORCID

Luana Martins https://orcid.org/0000-0001-6340-7615
Heitor Costa https://orcid.org/0000-0002-9903-7414
Ivan Machado https://orcid.org/0000-0001-9027-2293

ENDNOTES

* Available at: https://github.com/apache/camel/.

† Available at: https://github.com/arieslab/TSSM.

‡ Available at: https://github.com/arieslab/jnose.

§ Available at: https://github.com/mauricioaniche/ck.

¶ Available at: https://github.com/.

https://www.evosuite.org/.

‖ https://randoop.github.io/randoop/.

** https://sites.google.com/site/saktiabdel/JTExpert.

†† Available at: https://github-api.kohsuke.org/apidocs/index.html.

‡‡ Available at: https://bit.ly/3wiiNNc.

§§ Available at: https://bit.ly/3Cq4Rod.

¶¶ Available at: https://bit.ly/3Advji3.

REFERENCES

1. Bladel B, Demeyer S. Test behaviour detection as a test refactoring safety. In: *2nd International Workshop on Refactoring*. New York, NY, USA: ACM Association for Computing Machinery; 2018:22-25.
2. Garousi V, Kucuk B, Felderer M. What we know about smells in software test code. *IEEE Softw*. 2019;36(3):61-73.
3. Palomba F, Zaidman A, De Lucia A. Automatic test smell detection using information retrieval techniques. In: *International Conference on Software Maintenance and Evolution*. IEEE Institute of Electrical and Electronics Engineers; 2018:311-322.
4. Spadini D, Palomba F, Zaidman A, Bruntink M, Bacchelli A. On the relation of test smells to software code quality. In: *International Conference on Software Maintenance and Evolution*. IEEE Institute of Electrical and Electronics Engineers; 2018:1-12.
5. Palomba F, Di Nucci D, Panichella A, Oliveto R, De Lucia A. On the diffusion of test smells in automatically generated test code: An empirical study. In: *International Workshop on Search-Based Software Testing*. NY, USA: ACM Association for Computing Machinery; 2016:5-14.
6. Tufano M, Palomba F, Bavota G, Penta MD, Oliveto R, Lucia AD, Poshyvanyk D. An empirical investigation into the nature of test smells. In: *31st International Conference on Automated Software Engineering*. ACM Association for Computing Machinery; 2016:4-15.
7. Meszaros G, Smith SM, Andrea J. The test automation manifesto. *Extreme Programming and Agile Methods - XP/Agile Universe 2003*. Springer; 2003:73-81.
8. Deursen A, Moonen LM, Bergh A, Kok G. Refactoring test code, NLD, Centre for Mathematics and Computer Science; 2001.

9. Junior NS, Martins LA, Rocha L, Costa HAX, Machado I. How are test smells treated in the wild? A tale of two empirical studies. *J Softw Eng Res Development*. 2022;9:9:1-9:16.
10. Spadini D, Schvarcbacher M, Oprescu A-M, Bruntink M, Bacchelli A. Investigating severity thresholds for test smells. In: *17th International Conference on Mining Software Repositories*. ACM Association for Computing Machinery; 2020:311-321.
11. Bavota G, Qusef A, Oliveto R, Lucia AD, Binkley DW. Are test smells really harmful? an empirical study. *Empir Softw Eng*. 2015;20(4):1052-1094.
12. Peruma A, Almalki K, Newman CD, Mkaouer MW, Ouni A, Palomba F. On the distribution of test smells in open source android applications: An exploratory study. In: *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering IBM Corp.*; USA; 2019: 193-202.
13. Van Rompaey B, Du Bois B, Demeyer S, Rieger M. On the detection of test smells: A metrics-based approach for general fixture and eager test. *IEEE Trans Softw Eng*. 2007;33(12):800-817.
14. Guerra EM, Fernandes CT. Refactoring test code safely. In: International Conference on Software Engineering Advances. IEEE Institute of Electrical and Electronics Engineers; 2007:44-44.
15. Bavota G, Qusef A, Oliveto R, Lucia AD, Binkley DW. An empirical analysis of the distribution of unit test smells and their impact on software maintenance. In: 28th International Conference on Software Maintenance. IEEE Institute of Electrical and Electronics Engineers; 2012:56-65.
16. Virgínio T, Martins LA, Soares LR, Santana R, Costa H, Machado I. An empirical study of automatically-generated tests from the perspective of test smells. In: *Brazilian Symposium on Software Engineering*. ACM Association for Computing Machinery; 2020:92-96.
17. Tahir A, Counsell S, MacDonell SG. An empirical study into the relationship between class features and test smells. In: *23rd Asia-Pacific Software Engineering Conference*. IEEE Institute of Electrical and Electronics Engineers; 2016:137-144.
18. Virgnio T, Martins L, Santana R, et al. On the test smells detection: an empirical study on the jnose test accuracy. *J Softw Eng Res Development*. 2021; 9(1):1-14.
19. Peruma A, Almalki K, Newman CD, Mkaouer MW, Ouni A, Palomba F. tsdetect: an open source test smells detection tool. In: Symposium on the Foundations of Software Engineering. New York, NY, USA: ACM Association for Computing Machinery; 2020:1650-1654.
20. Santana R, Martins L, Rocha L, et al. Raide: A tool for assertion roulette and duplicate assert identification and refactoring. In: *34th Brazilian Symposium on Software Engineering*. New York, NY, USA: ACM Association for Computing Machinery; 2020:374-379.
21. Greiler M, Van Deursen A, Storey M-A. Automated detection of test fixture strategies and smells. In: *Sixth International Conference on Software Testing, Verification and Validation*. IEEE Institute of Electrical and Electronics Engineers; 2013:322-331.
22. Van Rompaey B, Du Bois B, Demeyer S. Characterizing the relative significance of a test smell. In: *22nd IEEE International Conference on Software Maintenance*. IEEE Institute of Electrical and Electronics Engineers; 2006:391-400.
23. Virgínio T, Martins L, Rocha L, Santana R, Cruz A, Costa H, Machado I. Jnose: Java test smell detector. In: *34th Brazilian Symposium on Software Engineering*. ACM Association for Computing Machinery; 2020:564-569.
24. Aniche M. Java code metrics calculator (ck). Available in https://github.com/mauricioaniche/ck/; 2022.
25. Virgínio T, Santana R, Martins LA, Soares LR, Costa H, Machado I. On the influence of test smells on test coverage. In: *XXXIII Brazilian Symposium on Software Engineering*. ACM Association for Computing Machinery; 2019:467-471.
26. Pecorelli F, Catolino G, Ferrucci F, De Lucia A, Palomba F. Testing of mobile applications in the wild: A large-scale empirical study on android apps. In: *28th International Conference on Program Comprehension*. New York, NY, USA: ACM Association for Computing Machinery; 2020:296-307.
27. Grano G, Palomba F, Di Nucci D, De Lucia A, Gall HC. Scented since the beginning: On the diffuseness of test smells in automatically generated test code. *J Syst Softw*. 2019;156:312-327.
28. Panichella A, Panichella S, Fraser G, Sawant AA, Hellendoorn VJ. Test smells 20 years later: Detectability, validity, and reliability. *Empir Softw Eng*. 2022;27(7):1-40.
29. Baker P, Evans D, Grabowski J, Neukirchen H, Zeiss B. Trex - the refactoring and metrics tool for ttcn-3 test specifications. In: *Testing: Academic Industrial Conference, Practice And Research Techniques*. ACM Association for Computing Machinery; 2006:90-94.
30. Zeiss B, Neukirchen H, Grabowski J, Evans D, Baker P. Refactoring and metrics for ttcn-3 test suites. In: *5th International Conference on System Analysis and Modeling: Language Profiles*. Berlin, Heidelberg: Springer Berlin Heidelberg Springer; 2006:148-165.
31. Counsell S, Hierons RM. Refactoring test suites versus test behaviour: A ttcn-3 perspective. In: *International Workshop on Software Quality Assurance*. ACM Association for Computing Machinery; 2007:31-38.
32. Gatrell M, Counsell S, Hall T. Empirical support for two refactoring studies using commercial c# software. In: *13th International Conference on Evaluation and Assessment in Software Engineering*. New York, NY, USA: ACM Association for Computing Machinery; 2009:1-10.
33. De Bleser J, Di Nucci D, De Roover C. Assessing diffusion and perception of test smells in scala projects. In: *16th International Conference on Mining Software Repositories*. IEEE Press; 2019:457-467.
34. Fernandes D, Machado I, Maciel R. Handling test smells in python: Results from a mixed-method study. In: *Brazilian Symposium on Software Engineering*. ACM Association for Computing Machinery; 2021:84-89.
35. Jorge D, Machado P, Andrade W. Investigating test smells in javascript test code. In: *Brazilian Symposium on Systematic and Automated Software Testing*. ACM Association for Computing Machinery; 2021:36-45.
36. Santana R, Fernandes D, Campos D, Soares L, Maciel R, Machado I. Understanding practitioners strategies to handle test smells: A multi-method study. In: *Brazilian Symposium on Software Engineering Association for Computing Machinery*; 2021:49-53.
37. Qusef A, Elish MO, Binkley D. An exploratory study of the relationship between software test smells and fault-proneness. *IEEE Access*. 2019;7: 139526-139536.
38. Kim DJ, Chen T-HP, Yang J. The secret life of test smells-an empirical study on test smell evolution and maintenance. *Empir Softw Eng*. 2021;26(5): 1-47.
39. Soares E, Ribeiro M, Amaral G, Gheyi R, Fernandes L, Garcia A, Fonseca B, Santos A. Refactoring test smells: A perspective from open-source developers. In: *5th Brazilian Symposium on Systematic and Automated Software Testing*. New York, NY, USA: ACM Association for Computing Machinery; 2020:50-59.
40. Soares E, Ribeiro M, Gheyi R, Amaral G, Santos AM. Refactoring test smells with junit 5: Why should developers keep up-to-date. *IEEE Trans Softw Eng*. 2022;2022:1-1.
41. Garousi V, Kucuk B. Smells in software test code: A survey of knowledge in industry and academia. *J Syst Softw*. 2018;138:52-81.

42. Aljedaani W, Peruma A, Aljohani A, et al. Test smell detection tools: A systematic mapping study. In: *Evaluation and Assessment in Software Engineering Association for Computing Machinery*; 2021:170-180.
43. Greiler M, Zaidman A, van Deursen A, Storey M-A. Strategies for avoiding text fixture smells during software evolution. In: 10th Working Conference on Mining Software Repositories. IEEE Institute of Electrical and Electronics Engineers; 2013:387-396.
44. Lambiase S, Cupito A, Pecorelli F, De Lucia A, Palomba F. Just-in-time test smell detection and refactoring: The darts project. In: *28th International Conference on Program Comprehension*. ACM Association for Computing Machinery; 2020:441-445.
45. Martinez M, Etien A, Ducasse S, Fuhrman C. Rtj: A java framework for detecting and refactoring rotten green test cases. In: *42nd International Conference on Software Engineering*. New York, NY, USA: ACM Association for Computing Machinery; 2020:69-72.
46. Koochakzadeh N, Garousi V. A tester-assisted methodology for test redundancy detection. *Adv Softw Eng*. 2010;2010(6):1-13.
47. Reichhart S, Gîrba T, Ducasse S. Rule-based assessment of test quality. *J Object Technol*. 2007;6(9):231-251.
48. Martins L, Bezerra C, Costa H, Machado I. Smart prediction for refactorings in the software test code. In: *XXXV Brazilian Symposium on Software Engineering*. NY, USA: ACM Association for Computing Machinery; 2021:115-120.
49. Hadj-Kacem M, Bouassida N. A multi-label classification approach for detecting test smells over java projects. *J King Saud Univers-Comput Inf Sci*. 2021;34(10):8692-8701.
50. Loriot B, Madeiral F, Monperrus M. Styler: Learning formatting conventions to repair checkstyle errors. arXiv, Available at: https://arxiv.org/abs/1904.01754; 2022.
51. Durieux T, Soto-Valero C, Baudry B. DUETS: A dataset of reproducible pairs of java library-clients. In: *18th International Conference on Mining Software Repositories*. IEEE Institute of Electrical and Electronics Engineers; 2021:545-549.
52. Fogel K. *Producing open source software: How to run a successful free software project*. 2nd ed., O'Reilly Media, Inc.; 2005.
53. Gold NE, Krinke J. Ethics in the mining of software repositories. *Empir Softw Eng*. 2022;27(1):1-49.
54. Jarczyk O, Gruszka B, Jaroszewicz S, Bukowski L, Wierzbicki A. Github projects. quality analysis of open-source software. In: *International Conference on Social Informatics*. Springer Springer, Cham; 2014:80-94.
55. Chidamber SR, Kemerer CF. A metrics suite for object oriented design. *IEEE Trans Softw Eng*. 1994;20(6):476-493.
56. Zhou Y, Leung H, Xu B. Examining the potentially confounding effect of class size on the associations between object-oriented metrics and change-proneness. *IEEE Trans Softw Eng*. 2009;35(5):607-623.
57. Chowdhury SA, Uddin G, Holmes R. An empirical study on maintainable method size in java. In: *19th International Conference on Mining Software Repositories*. IEEE Institute of Electrical and Electronics Engineers; 2022:252-264.
58. Kochhar PS, Bissyandé TF, Lo D, Jiang L. Adoption of software testing in open source projects–a preliminary study on 50,000 projects. In: *17th european conference on software maintenance and reengineering*. IEEE Institute of Electrical and Electronics Engineers; 2013:353-356.
59. Hauke J, Tomasz K. Comparison of values of pearson's and spearman's correlation coefficients on the same sets of data. *Quaest Geographicae*. 2011;30(2):87-93.
60. Salkind NJ, Rainwater T. *Exploring research*. 8th ed. Pearson; 2012.
61. Theodorsson-Norheim E. Kruskal-Wallis test: Basic computer program to perform nonparametric one-way analysis of variance and multiple comparisons on ranks of several independent samples. *Comput Methods Programs Biomed*. 1986;23(1):57-62.

How to cite this article: Martins L, Costa H, Machado I. On the diffusion of test smells and their relationship with test code quality of Java projects. *J Softw Evol Proc*. 2024;36(4):e2532. doi:10.1002/smr.2532